

Deep Q-Networks (DQN)

First, a story

In 2013, DeepMind published a paper doing something that sounded crazy: with one codebase and one set of hyperparameters, let an AI play Atari games. Nobody told it the rules — only screen pixels and the score, like a kid with no manual mashing buttons in front of the TV.

By 2015, the upgraded version beat most of 49 games, matching or exceeding human level on Breakout, Pong and others. This AI is **DQN (Deep Q-Network)**.

Before DQN, a recognized chasm separated deep neural networks from reinforcement learning: **training instability**. Approximating Q with a network often diverged. DQN closed it with two elegant engineering tricks.

The core question of this chapter: Q-learning already works — why does it need DQN?

Why a Q Table Is Not Enough — the Curse of Dimensionality

Recall the Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

The algorithm needs a **Q table**: one number per state-action pair (s, a) . In a discrete grid world that is fine. In CartPole the state is 4 continuous numbers (cart position, velocity, pole angle, angular velocity) — the table would have infinitely many rows.

Discretizing does not save you:

Bins per dimension	CartPole Q-table entries (4 dims, 2 actions)
$b = 10$	$10^4 \times 2 = 20,000$
$b = 20$	$20^4 \times 2 = 320,000$
$b = 100$	$100^4 \times 2 = 200,000,000$

An Atari state is an $84 \times 84 \times 4$ stack of grayscale frames — about $256^{84 \times 84 \times 4}$ table entries, beyond writing down (Figure 1).

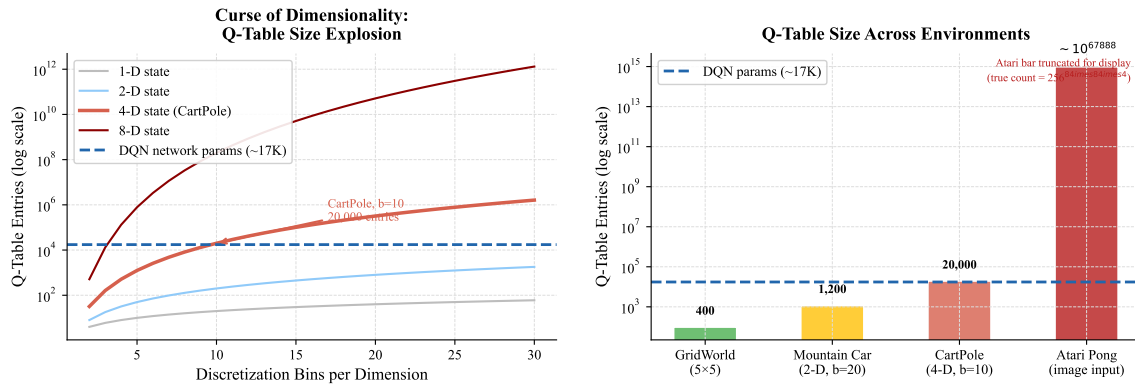


Figure 1: The curse of dimensionality: Q-table size explodes exponentially with state dimension and discretization resolution, far beyond the parameter count of a small neural network.

The way out: stop storing a table; fit $Q(s, a)$ with a **function**.

Innovation 1: Approximating Q with a Neural Network

From Q table to Q network

Replace the table by a network with parameters θ :

$$Q(s, a; \theta) \approx Q^*(s, a)$$

The input is the state s ; the output is **the vector of Q values for all actions** — one forward pass yields every action’s estimate:

$$a^* = \arg \max_a Q(s, a; \theta)$$

The loss

Training pulls predictions toward Bellman targets with a squared loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Here $\mathcal{D}$ is the experience dataset and θ^- the target-network parameters (detailed below).

Two fatal problems of naive training

Naively descending on each transition (s, a, r, s') hits:

- **Problem 1 — temporal correlation:** consecutive frames are nearly identical; gradient estimates have high variance and training oscillates;
- **Problem 2 — the moving target:** the target $r + \gamma \max_{a'} Q(s', a'; \theta)$ itself moves with θ — chasing a moving target, Q values easily diverge.

DQN’s other two innovations each address one of these precisely.

Innovation 2: Experience Replay

The problem: temporal correlation

Online Q-learning updates the network after every step. Adjacent states are highly correlated (cart leaning right $\rightarrow$ next frame still leaning right), violating the i.i.d. assumption behind stochastic gradient descent: gradient variance blows up and training oscillates.

The fix: a replay buffer

Store every transition $(s, a, r, s', \text{done})$ in a FIFO **replay buffer** $\mathcal{D}$ (capacity N); at training time sample a mini-batch **uniformly at random**:

$$\{(s_i, a_i, r_i, s'_i)\}_{i=1}^B \sim \text{Uniform}(\mathcal{D})$$

Why it works

- **Breaks temporal correlation:** random sampling makes batches approximately i.i.d., restoring the SGD assumption;
- **Improves data efficiency:** each experience can be sampled many times;
- **Stabilizes gradients:** larger batch B means lower gradient variance.

The ablation in Figure 2 shows it: without replay, training curves have much larger variance and slower convergence.

Innovation 3: The Target Network

The problem: the moving target

In naive DQN both sides of the loss contain θ :

$$\mathcal{L}(\theta) = \mathbb{E} \left[\underbrace{\left(r + \gamma \max_{a'} Q(s', a'; \theta) \right)}_{\text{target}} - \underbrace{Q(s, a; \theta)}_{\text{prediction}} \right]^2$$

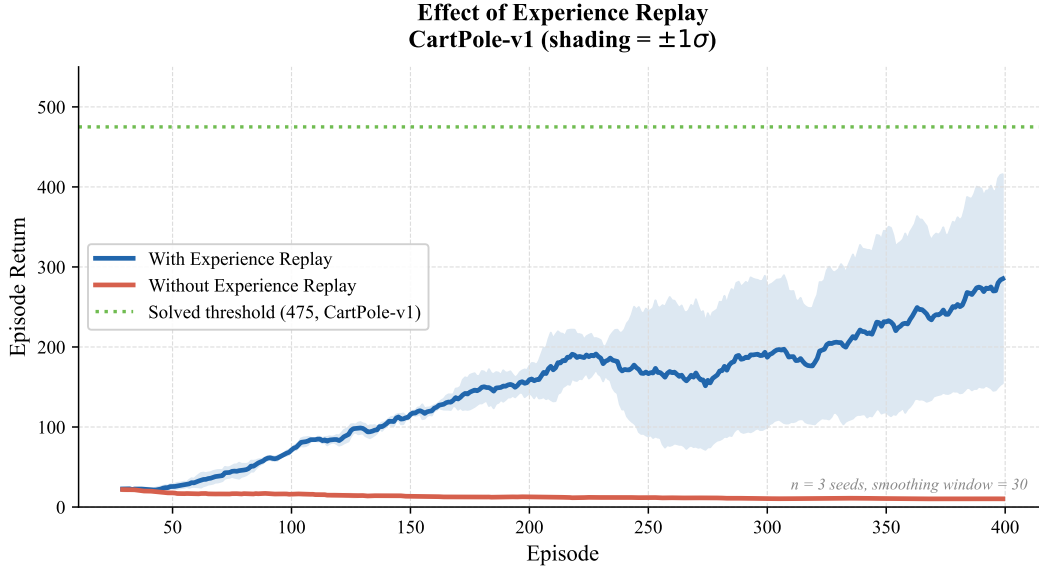


Figure 2: Experience-replay ablation (CartPole-v1, 3 random seeds, shading $\pm 1\sigma$). With replay the run converges faster with less variance.

Every gradient step moves the prediction toward the target — but the target moves with θ too. This is **chasing a moving target**: updating one Q value indirectly drags others, triggering cascading instability until the values diverge.

The fix: a frozen target network

Maintain two networks:

- **Online network** $Q(s, a; \theta)$: updated by gradient descent every step, used to choose actions;
- **Target network** $Q(s, a; \theta^-)$: **frozen**; it only enters Bellman targets and is hard-synced from the online network every C steps:

$$\theta^- \leftarrow \theta \quad (\text{every } C \text{ steps; frozen in between})$$

The loss becomes:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta^-)}_{\text{fixed target (short-term)}} - Q(s, a; \theta) \right)^2 \right]$$

Intuition: swap “a target that moves every step” for “one that shifts every C steps” — over the short term the target is static and training is far more stable. Larger C means a steadier but more stale θ^- ; the value is task-dependent — the Atari paper used $C = 10^4$, small tasks typically $10^2 \sim 10^3$.

The ablation is shown in Figure 3.

The Full DQN Algorithm

Combining the three innovations:

1. **Initialize**: replay buffer $\mathcal{D}$ (capacity N), online parameters θ , target $\theta^- \leftarrow \theta$, exploration rate ε ;
2. **Each episode**, for each step t :
 - (a) choose ε -greedily:

$$a_t = \begin{cases} \text{random} & \text{with probability } \varepsilon \\ \arg \max_a Q(s_t, a; \theta) & \text{with probability } 1 - \varepsilon \end{cases}$$

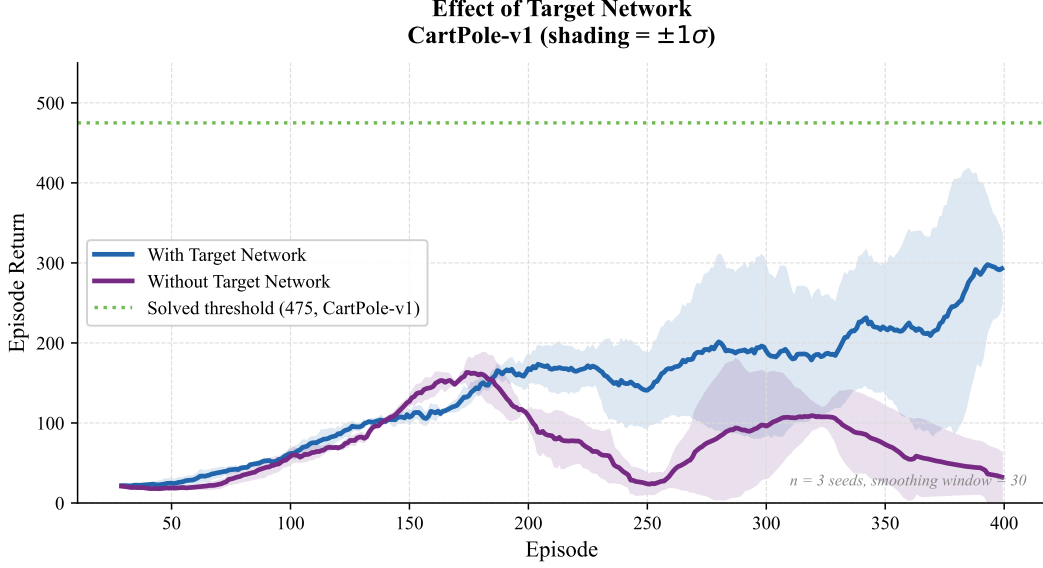


Figure 3: Target-network ablation (CartPole-v1, 3 random seeds, shading $\pm 1\sigma$). Without a target network the training curve is less stable with higher variance across seeds.

- (b) execute a_t , observe r_t, s_{t+1} ; store $(s_t, a_t, r_t, s_{t+1}, \text{done})$ in $\mathcal{D}$;
- (c) sample B transitions uniformly; compute Bellman targets:

$$y_i = \begin{cases} r_i & \text{if terminal} \\ r_i + \gamma \max_{a'} Q(s'_i, a'; \theta^-) & \text{otherwise} \end{cases}$$

- (d) gradient step: $\theta \leftarrow \theta - \alpha \nabla_{\theta} \frac{1}{B} \sum_{i=1}^B (y_i - Q(s_i, a_i; \theta))^2$;

- (e) every C steps: $\theta^- \leftarrow \theta$.

The three innovations in one line each:

Problem	Innovation	Operation
unbounded state space	network approximation	$Q(s, a; \theta) \approx Q^*(s, a)$
temporally correlated samples	experience replay	$(s, a, r, s') \sim \text{Uniform}(\mathcal{D})$
unstable targets	target network	$\theta^- \leftarrow \theta$ every C steps

Where DQN sits in the RL landscape

DQN is the starting point of deep RL, inheriting classic RL and opening the deep-RL mainline:

- From tables to networks:** Q-learning (tabular, small discrete states) $\rightarrow$ DQN (network, continuous high-dimensional states); the update rule is identical, only the approximator changed;
- Replay's long shadow:** a fixed dataset plus offline learning is precisely the paradigm of **offline RL** — the data of later algorithms such as CQL and IQL all comes from some form of replay buffer;
- Evolution of the target network:** hard update $\theta^- \leftarrow \theta \rightarrow$ soft (Polyak) update $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$, adopted widely by DDPG, TD3 and SAC;
- Next steps:** Double DQN (decoupling action selection from evaluation to curb overestimation) and Dueling DQN (splitting Q into V plus advantage A) — both precise upgrades within the DQN framework.

References

1. Mnih, V., et al. (2013). Playing Atari with Deep Reinforcement Learning. *NeurIPS Deep Learning Workshop*.
2. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
3. Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). Chapters 9–10.
4. Hands-on RL (Chinese). Chapter 7: the DQN algorithm.